

S03 - Registro, Descubrimiento y Ejecución Concurrente de Servicios

Evidencia individual — pagatu-orden-ms

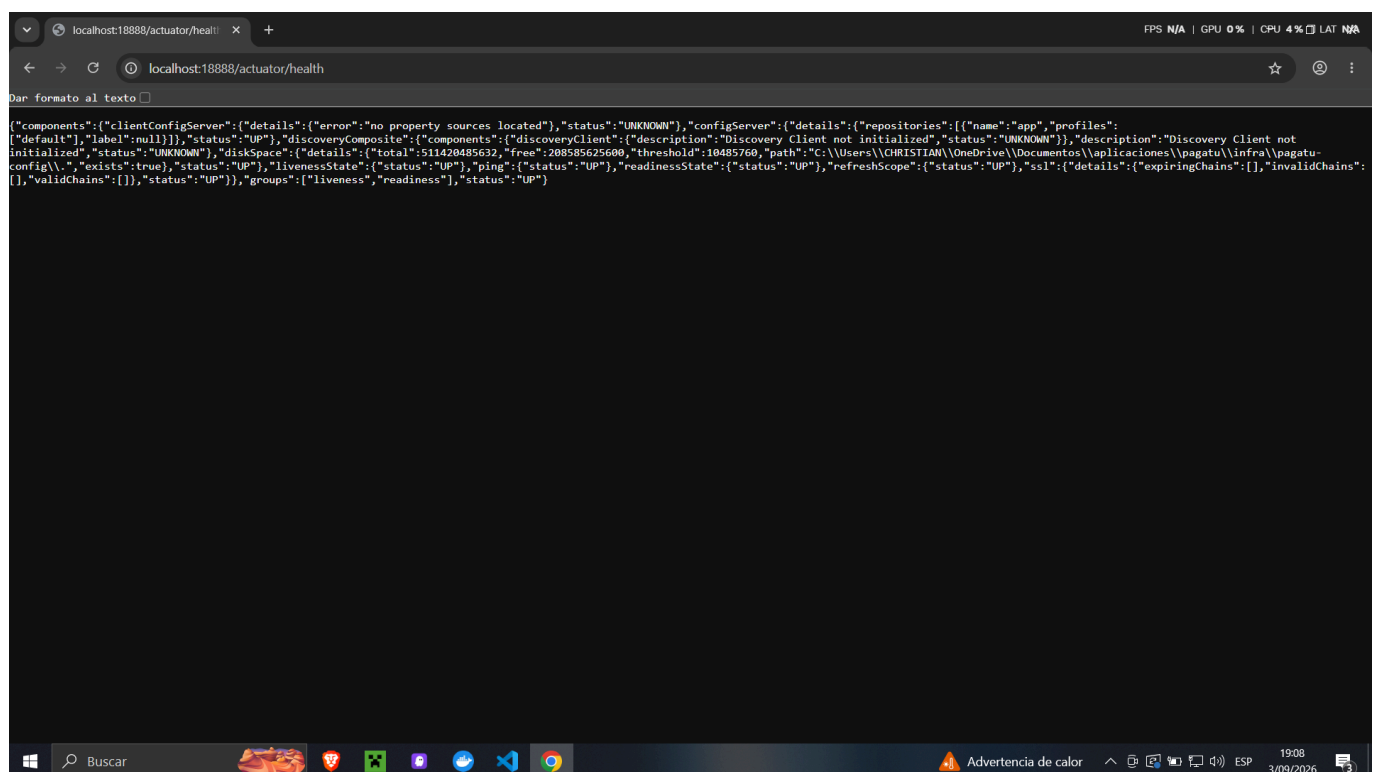
Datos del estudiante

- **Nombre:** Christian Yoel Soncco Vargas
- **Equipo:** sin equipo
- **Sesión:** S03 - Registro, Descubrimiento y Ejecución Concurrente de Servicios
- **Rol o aporte realizado:** Construcción de **pagatu-eureka** como servidor de registro y conexión de **pagatu-orden-ms** como cliente Eureka, con dos instancias simultáneas verificadas en el dashboard.
- **Link de GitHub:** <https://github.com/christianyael>

1. pagatu-eureka operativo

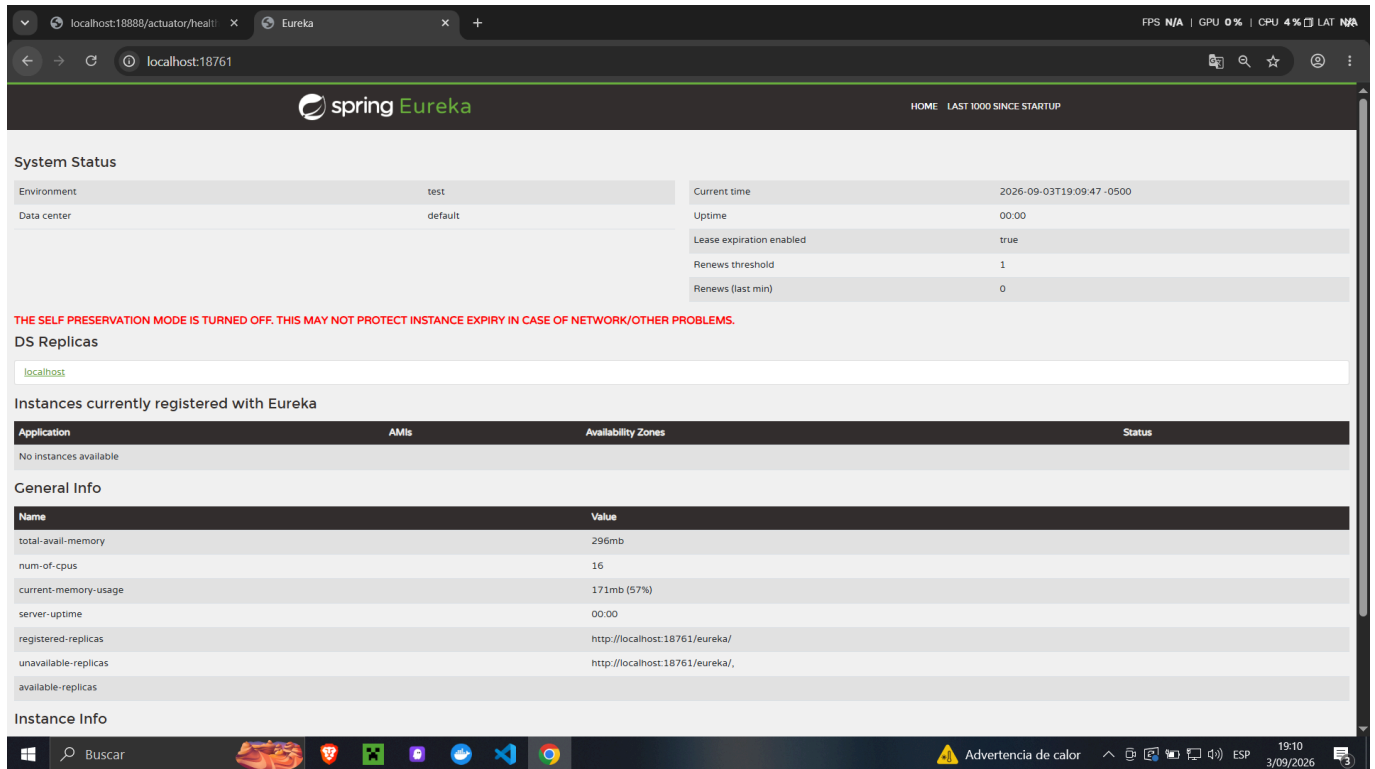
Se construyó **pagatu-eureka** dentro de **infra/pagatu-eureka** con las dependencias **spring-cloud-starter-netflix-eureka-server**, **spring-cloud-starter-config** y **spring-boot-starter-actuator**. La clase principal se anotó con **@EnableEurekaServer** para activar el servidor de registro. Su **application.yml** es mínimo y sigue el mismo patrón de S02: solo declara el nombre y delega toda la configuración a **pagatu-config**.

La configuración completa de DEV vive en **config-repo/pagatu-eureka-dev.yml** con **enable-self-preservation: false** (solo en DEV, para evitar alertas con pocas instancias) y **register-with-eureka: false** (el propio servidor no debe registrarse como cliente).

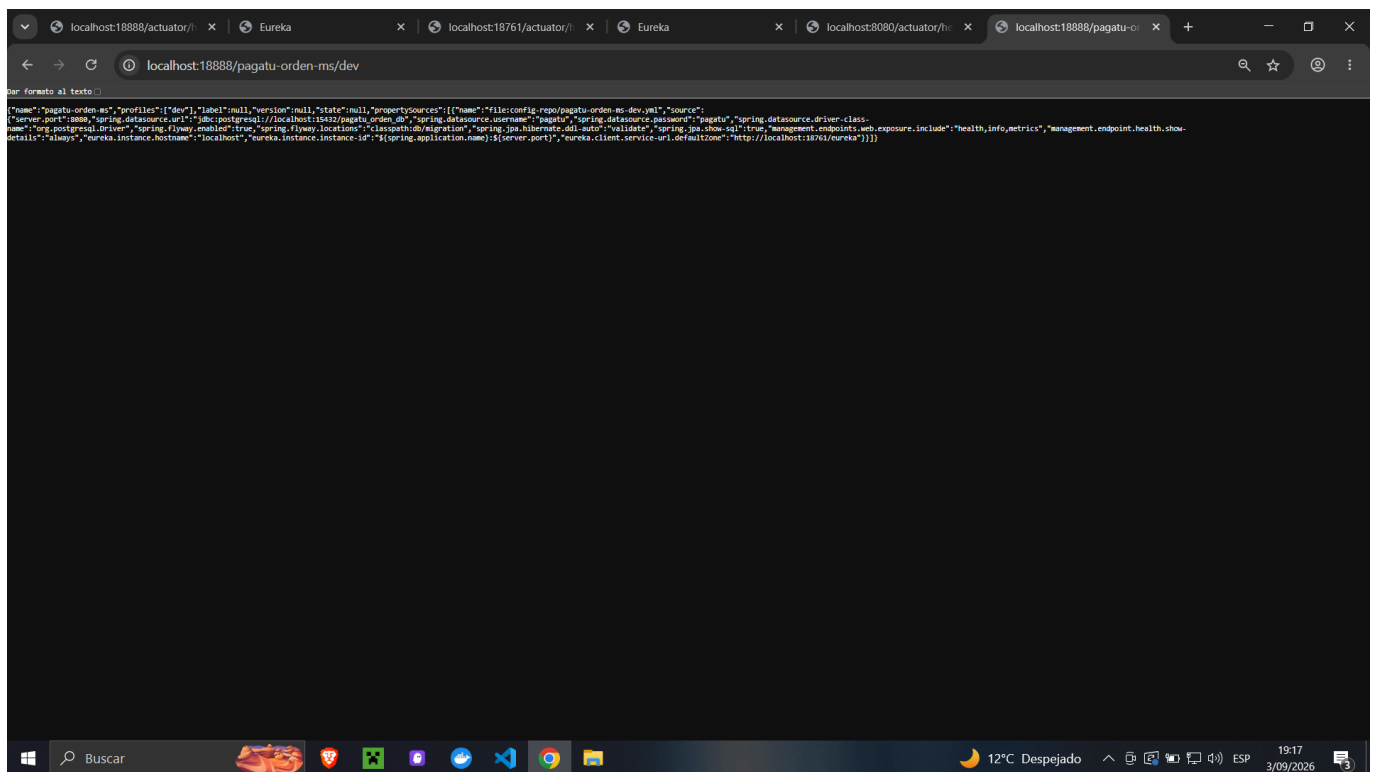


Descripción: **localhost:18888/actuator/health** muestra estado UP, confirmando que el Config Server está

operativo y que *pagatu-eureka* puede cargar su configuración al arrancar.



Descripción: Dashboard de *pagatu-eureka* en *Localhost:18761* recién arrancado, sin instancias registradas. El banner "THE SELF PRESERVATION MODE IS TURNED OFF" confirma que la configuración de DEV se aplicó correctamente.

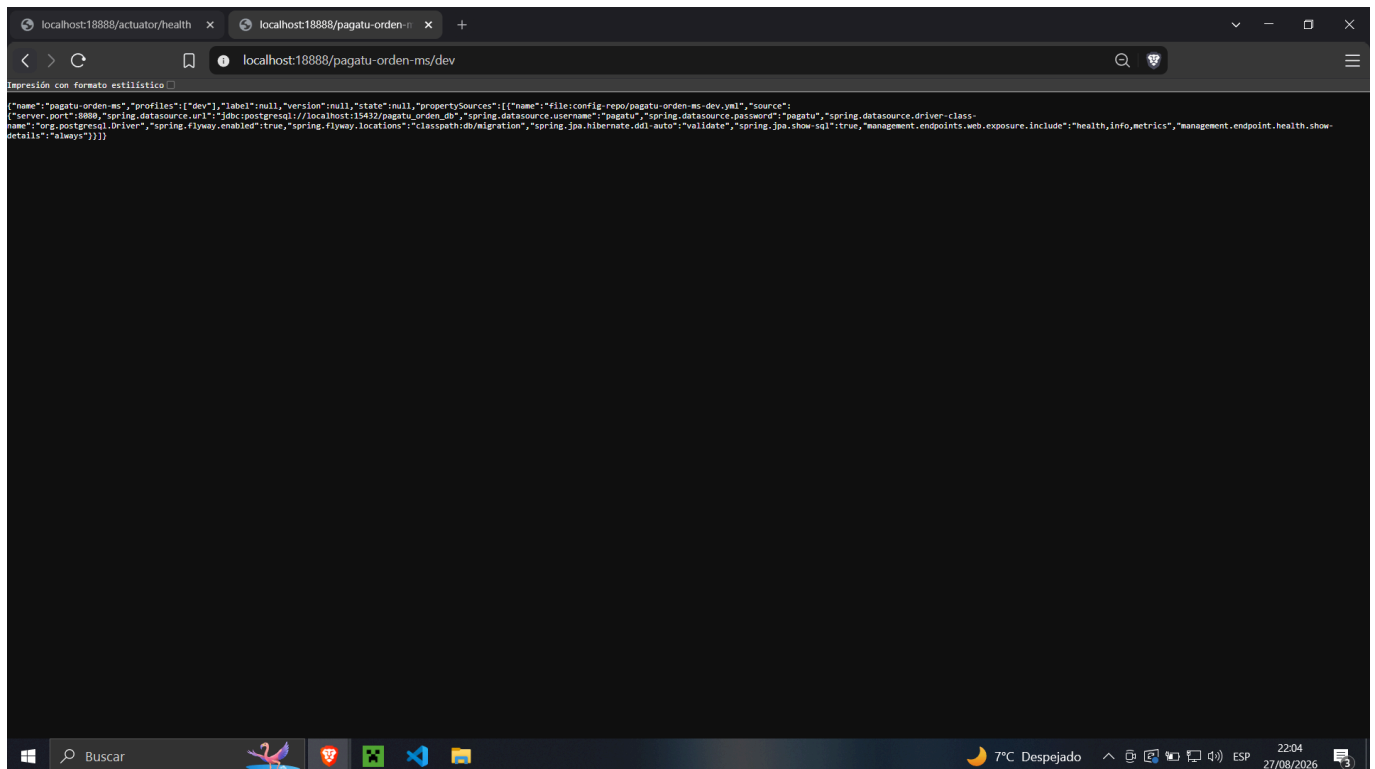


Descripción: *Localhost:18761/actuator/health* devuelve UP con *configServer* y *discoveryComposite* activos, confirmando que el servidor de registro está operativo y conectado a *pagatu-config*.

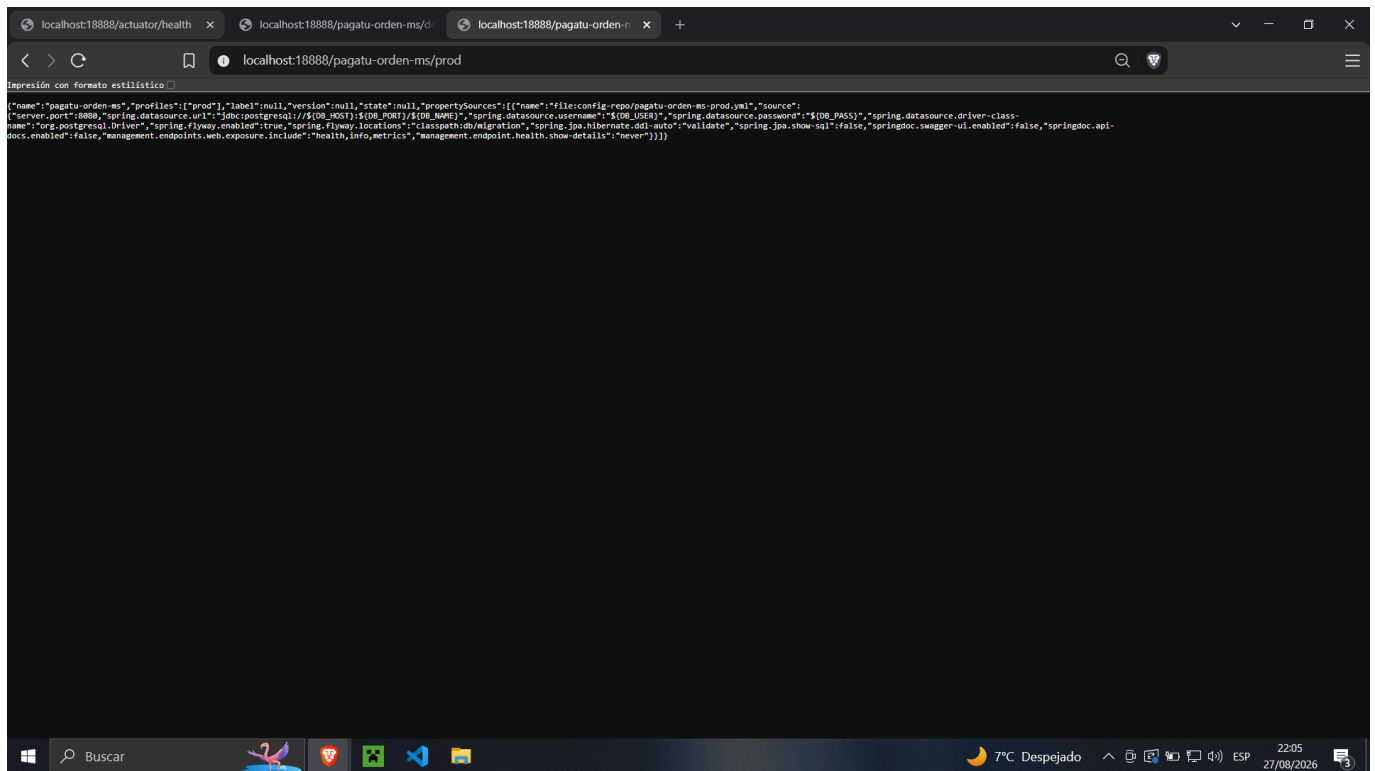
2. pagatu-orden-ms migrado a Config Client y conectado a Eureka

`pagatu-orden-ms` ya tenía Config Client desde S02. En esta sesión se agregó `spring-cloud-starter-netflix-eureka-client` al `pom.xml` y el bloque `eureka:` en `config-repo/pagatu-orden-ms-dev.yml` como sección raíz independiente, al mismo nivel que `server:` y `management::`:

```
eureka:
  instance:
    hostname: localhost
    instance-id: ${spring.application.name}:${server.port}
  client:
    service-url:
      defaultZone: http://localhost:18761/eureka
```



Descripción: `localhost:18888/pagatu-orden-ms/dev` devuelve el JSON con todas las propiedades de `pagatu-orden-ms-dev.yml`, incluyendo la sección `eureka` con `instance-id` y `defaultZone`. Confirma que `pagatu-config` entrega correctamente la configuración de Eureka Client.



Descripción: *localhost:18888/pagatu-orden-ms/prod* muestra las propiedades del ambiente de producción, evidenciando la separación real entre ambientes.

3. pagatu-orden-ms registrado con múltiples instancias

Se levantó primero el contenedor de PostgreSQL y luego la primera instancia del microservicio con `.\mvnw.cmd spring-boot:run`. Para la segunda instancia, sin cerrar la primera, se pasó un puerto distinto por línea de comandos igual que en S01.

![Base de datos corriendo](base de datos.PNG) Descripción: El contenedor *pagatu-postgres-orden-dev* está en estado Running, habilitando la conexión de *pagatu-orden-ms* a la base de datos.

![Logs arranque instancia 8080](pagatu-orden-ms instancia 1.PNG) Descripción: La terminal muestra *Registering application PAGATU-ORDEN-MS with eureka with status UP, registration status: 204 y Tomcat started on port 8080. La instancia 1 se registró exitosamente en Eureka.*

The screenshot shows the Eureka Dashboard at localhost:18761. The 'System Status' section indicates the environment is 'test', data center is 'default', and the current time is 2026-09-03T19:15:38 -0500. A warning message states: 'RENEWALS ARE LESSER THAN THE THRESHOLD. THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.' The 'DS Replicas' section shows 'localhost'. The 'Instances currently registered with Eureka' table lists one instance: PAGATU-ORDEN-MS with 1 AMIs and 1 Availability Zone, status 'UP (1) - pagatu-orden-ms:8080'. The 'General Info' section shows server uptime as 00:05 and registered replicas at http://localhost:18761/eureka/.

Environment	test	Current time	2026-09-03T19:15:38 -0500
Data center	default	Uptime	00:05
		Lease expiration enabled	true
		Renews threshold	3
		Renews (last min)	0

RENEWALS ARE LESSER THAN THE THRESHOLD. THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.

Application	AMIs	Availability Zones	Status
PAGATU-ORDEN-MS	n/a (1)	(1)	UP (1) - pagatu-orden-ms:8080

Name	Value
total-avail-memory	188mb
num-of-cpus	16
current-memory-usage	82mb (43%)
server-uptime	00:05
registered-replicas	http://localhost:18761/eureka/
unavailable-replicas	http://localhost:18761/eureka/
available-replicas	

Descripción: Dashboard de Eureka mostrando PAGATU-ORDEN-MS con UP (1) - pagatu-orden-ms:8080. La instancia se anunció sola al arrancar.

The screenshot shows the Eureka Dashboard at localhost:8080/actuator/health. The response is a JSON object indicating the instance is 'UP' and has successfully registered with Eureka. The JSON includes details about the instance's configuration, including the database (PostgreSQL), discovery client, and services.

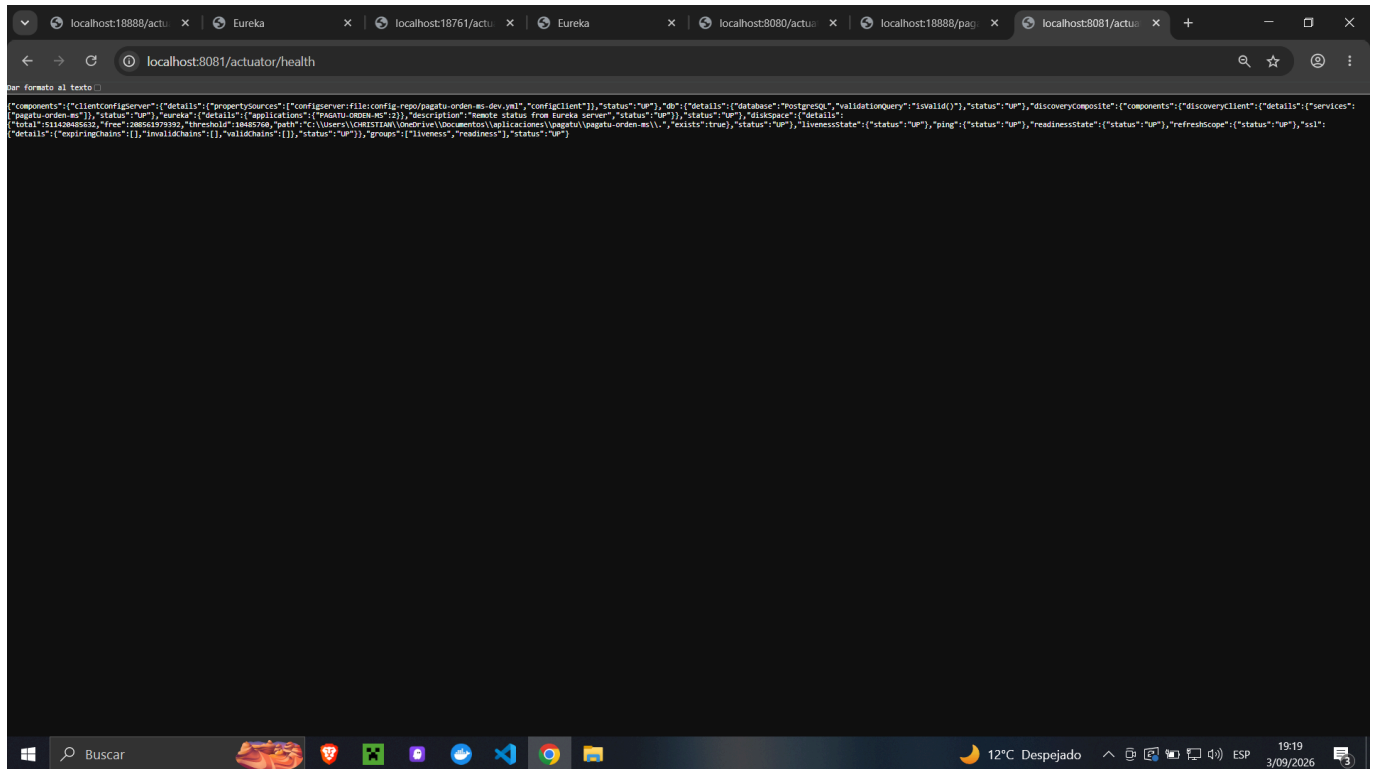
```
{
  "components": {
    "clientConfigServer": {
      "details": {
        "propertySource": "file:config-repo/pagatu-orden-ms-dev.yml",
        "configClient": true,
        "status": "UP",
        "db": {
          "details": {
            "database": "PostgreSQL",
            "validationQuery": "invalid()",
            "status": "UP",
            "discoveryComposite": {
              "components": {
                "discoveryClient": {
                  "details": {
                    "services": {
                      "pagatu-orden-ms": {
                        "status": "UP",
                        "eureka": {
                          "details": {
                            "applications": {
                              "PAGATU-ORDEN-MS": 1,
                              "description": "remote status from Eureka server",
                              "status": "UP",
                              "diskSpace": {
                                "details": {
                                  "total": 514384552,
                                  "free": 2857223696,
                                  "threshold": 18446760,
                                  "path": "C:\\Users\\CHRISTIAN\\OneDrive\\Documents\\aplicaciones\\pagatu\\pagatu-orden-ms\\",
                                  "exists": true,
                                  "status": "UP",
                                  "livenessState": {
                                    "status": "UP",
                                    "ping": {
                                      "status": "UP",
                                      "readinessState": {
                                        "status": "UP",
                                        "refreshScope": {
                                          "status": "UP",
                                          "ssl": {
                                            "details": {
                                              "expiryTime": 1,
                                              "invalidTime": 1,
                                              "validTime": 1,
                                              "status": "UP",
                                              "group": {
                                                "liveness": "readiness",
                                                "status": "UP"
                                              }
                                            }
                                          }
                                        }
                                      }
                                    }
                                  }
                                }
                              }
                            }
                          }
                        }
                      }
                    }
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Descripción: localhost:8080/actuator/health devuelve UP con db, eureka y configServer activos. La instancia 1 funciona correctamente con la configuración externa.

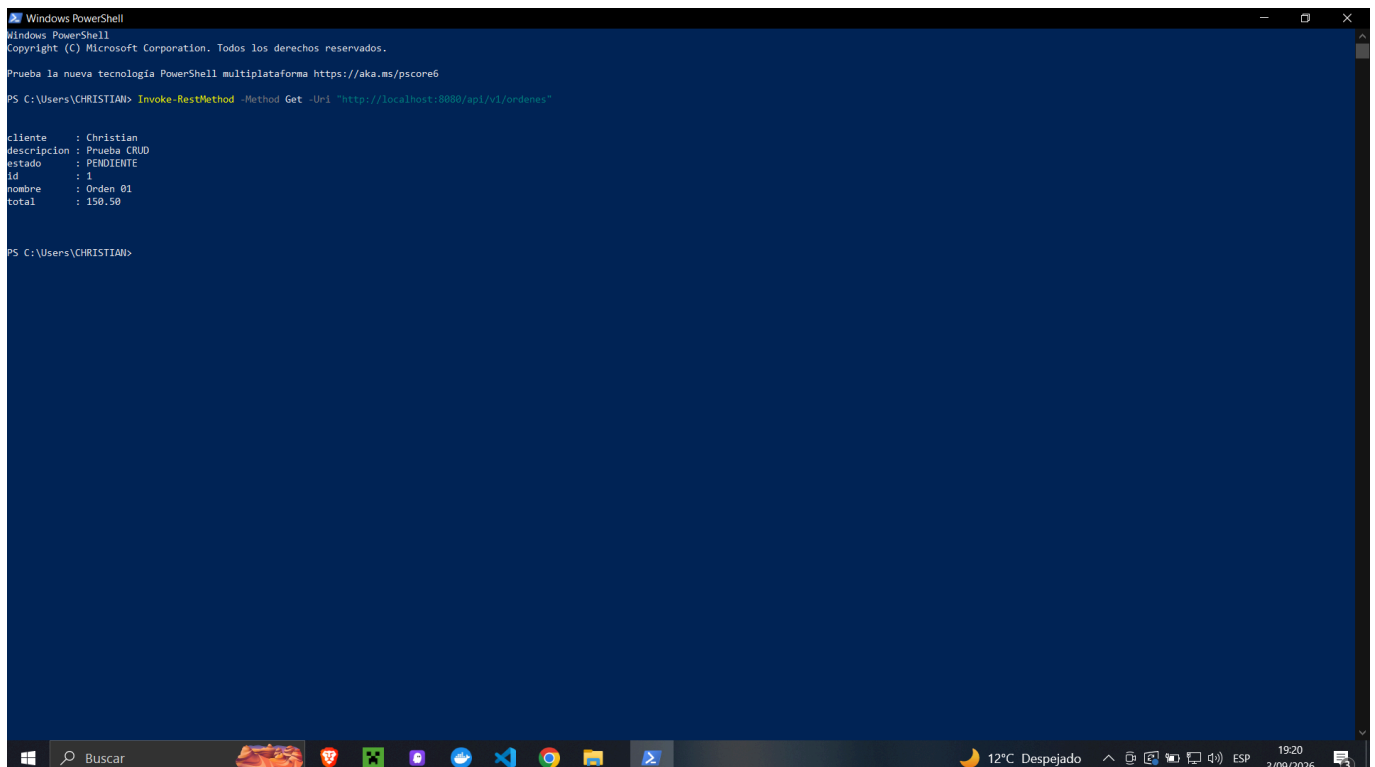
![Logs arranque instancia 8081](pagatu-orden-ms instancia 2.PNG) Descripción: La segunda terminal muestra registration status: 204 y Tomcat started on port 8081. La instancia 2 se registró en Eureka bajo el mismo nombre lógico con puerto distinto.

![Dashboard con 2 instancias](2 instancias.PNG) Descripción: Dashboard de Eureka mostrando PAGATU-ORDEN-MS con UP (2) - pagatu-orden-ms:8080, pagatu-orden-ms:8081. El mismo nombre lógico, dos

instancias con puertos fijos distintos, cada una registrada de forma independiente sin intervención manual.



*Descripción: **Localhost:8081/actuator/health** devuelve UP confirmando que la segunda instancia funciona de forma independiente.*



*Descripción: **Invoke-RestMethod** contra **Localhost:8080/api/v1/ordenes** devuelve la lista de órdenes. La instancia 1 responde correctamente.*

*! [GET ordenes puerto 8081] (power 8081.PNG) Descripción: El mismo endpoint contra **Localhost:8081/api/v1/ordenes** devuelve los mismos datos. Ambas instancias sirven tráfico de forma independiente desde la misma base de datos.*

![Instancia desaparece tras Ctrl+C](Detén una instancia y verifica.PNG) *Descripción: Tras detener la instancia 8081 con Ctrl+C, el dashboard de Eureka actualiza automáticamente sin intervención manual. La instancia desaparece del registro cuando expira su último heartbeat.*

4. Comprensión del patrón

Con dos instancias del mismo servicio en puertos distintos, cualquier cliente que quiera repartir tráfico entre ellas tiene que conocer ambas direcciones de antemano. Con varios microservicios y varias instancias cada uno, mantener esa lista manualmente se vuelve inmanejable.

El patrón Service Registry resuelve esto invirtiendo la responsabilidad: cada instancia se registra sola al arrancar con su nombre lógico y su dirección real. Quien necesite comunicarse pregunta al registro por el nombre, no por una dirección fija. Si una instancia cae, Eureka deja de anunciarla cuando expira su heartbeat, sin que nadie tenga que notificarlo.

En este proyecto, `pagatu-orden-ms` aparece como `PAGATU-ORDEN-MS` en el dashboard sin importar cuántas instancias estén corriendo ni en qué puertos. Los puertos `8080` y `8081` los conoce Eureka, no los clientes. Con el Gateway de S04 esto va un paso más allá: el cliente externo ni siquiera sabe que Eureka existe, llama siempre al mismo punto de entrada y el Gateway resuelve qué instancia atiende cada petición.

5. Error o hallazgo técnico

- **Qué ocurrió:** `pagatu-orden-ms` arrancaba en el puerto correcto pero no aparecía en el dashboard de Eureka.
 - **Cómo lo diagnosticué:** Revisé `config-repo/pagatu-orden-ms-dev.yml` y encontré que el bloque `eureka:` estaba indentado dentro de `management.endpoint.health` en lugar de ser una sección raíz. Spring Boot lo ignoraba silenciosamente como propiedad desconocida.
 - **Cómo lo corregí:** Moví `eureka:` al nivel raíz del YAML, al mismo nivel que `server:`, `spring:` y `management:`. Tras reiniciar `pagatu-config`, `pagatu-orden-ms` se registró correctamente en el siguiente arranque.
-

6. Reflexión técnica breve

¿Por qué el registro y descubrimiento de servicios es un prerrequisito para el Gateway y el balanceo de carga de S04?

El Gateway usa la dirección lógica `lb://pagatu-orden-ms` para enrutar peticiones y es Eureka quien resuelve esa dirección a las instancias reales disponibles en ese momento. Sin un registro operativo con los servicios registrados, el Gateway no tiene forma de saber a dónde enviar el tráfico. El registro es la fuente de verdad del sistema sobre qué instancias existen y están vivas; el Gateway es su consumidor más visible.

Anexo — Feedback de la sesión

1. El aprendizaje más importante es que cada instancia se registra sola y el cliente solo necesita el nombre lógico, no la dirección.

2. Lo más confuso fue entender por qué quien prueba a mano sigue necesitando el puerto exacto hoy, y cómo eso cambia con el Gateway en S04.
3. Para la siguiente clase: ¿el Gateway puede enrutar a servicios que no están en Eureka, o todo el tráfico tiene que pasar por el registro?
4. Nivel de comprensión: Entendido, podría explicarlo.
5. Para comprender mejor: un diagrama de secuencia del flujo completo desde una petición externa hasta la instancia que la atiende, pasando por el Gateway y Eureka.
6. Autoevaluación: Muy comprometido, me esforcé al máximo.
7. Satisfacción con la clase: 10.